TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 10

Security, GitOps and Production

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 10: Security, GitOps and production Kubernetes . . . 3

10.1 Who can do what? (RBAC) . . . 3

10.2 Securing pods . . . 4

10.3 NetworkPolicies . . . 5

10.4 CI/CD and GitOps . . . 6

10.5 Kubernetes in production . . . 7

10.6 Live lab: security and GitOps . . . 7

10.7 Final project: WordPress + MySQL . . . 8

Class 10 - Top 20 interview questions . . . 10

K8S · CLASS 10 · 60-90 minutes

Security, GitOps and production Kubernetes

By the end you can secure a cluster with RBAC, pod security and NetworkPolicies, explain GitOps, and describe how Kubernetes runs in real companies.

What you will learn today

- Authentication, authorization (RBAC) and admission
- ServiceAccounts, Roles, ClusterRoles and bindings
- securityContext and Pod Security Admission
- NetworkPolicies
- CI/CD and GitOps with Argo CD
- Managed Kubernetes, high availability, service mesh and cost
- Final project: deploy WordPress + MySQL end to end

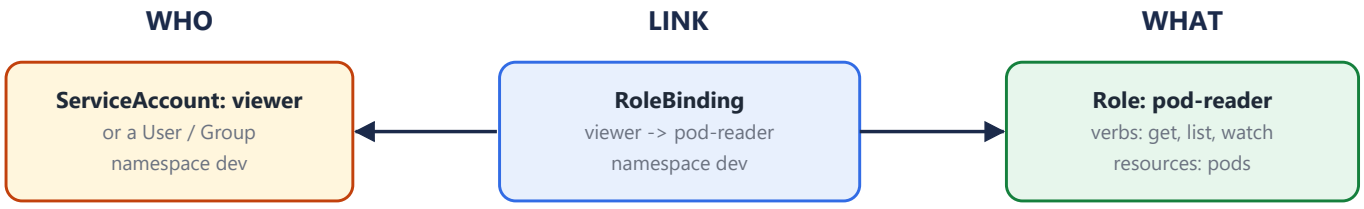
Part	Topic	Time
1	Story, RBAC and pod security	25 min
2	NetworkPolicies	15 min
3	CI/CD, GitOps and production topics	15 min
4	Live lab and final project brief	20 min
5	Recap and interview questions	10 min

10.1 Who can do what? (RBAC)

Ravi's story

An intern ran `kubectl delete namespace prod` thinking he was on his test cluster. He had full admin rights because everyone shared one kubeconfig. QuickBite was down for an hour. Ravi was asked to make sure "people and apps get only the access they need".

Every API request passes three gates: **authentication** (who are you? - certificates, tokens, cloud IAM, OIDC), **authorization** (are you allowed? - RBAC) and **admission control** (is the request acceptable? - quotas, Pod Security, policy engines).



ClusterRole + ClusterRoleBinding = the same idea for the whole cluster

Default is DENY - you only get what a binding gives you

Figure 10.1 - RBAC connects a subject to a set of permissions through a binding

Object	Scope	Purpose
Role	Namespace	Permissions (verbs on resources) inside one namespace
ClusterRole	Cluster	Permissions cluster-wide or for cluster-scoped resources; reusable in namespaces
RoleBinding	Namespace	Grants a Role or ClusterRole to subjects within one namespace
ClusterRoleBinding	Cluster	Grants a ClusterRole across the whole cluster
ServiceAccount	Namespace	Identity for pods/apps talking to the API

Everyday example
Office ID cards. Your card (ServiceAccount or user) says who you are. The access policy (Role) says "can open the 3rd-floor store room". The HR register entry (RoleBinding) links your card to that policy. Without an entry, the door stays shut - default deny.

rbac.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: viewer, namespace: dev }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: viewer-reads-pods, namespace: dev }
subjects:
  - kind: ServiceAccount
    name: viewer
    namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

10.2 Securing pods

Hardened securityContext

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
    seccompProfile: { type: RuntimeDefault }
  containers:
    - name: app
      image: my-api:2.1
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
```

Pod Security Admission enforces the Pod Security Standards per namespace with a label: `privileged` (anything), `baseline` (blocks known escalations) or `restricted` (best practice). Example: `kubectl label ns prod pod-security.kubernetes.io/enforce=restricted`. It replaced the old PodSecurityPolicy, removed in v1.25. For custom rules teams add **Kyverno** or **OPA Gatekeeper**.

Security layer	Good practice
Images	Small base images, pinned tags or digests, scanned with Trivy/Grype, signed with cosign
API access	RBAC least privilege, no shared admin kubeconfig, SSO/OIDC, audit logs
Pods	Non-root, read-only file system, drop capabilities, Pod Security `restricted`
Network	Default-deny NetworkPolicies, TLS everywhere, service mesh mTLS (Istio, Linkerd)
Secrets	Encryption at rest, external secret managers, no secrets in Git
Nodes & cluster	Patch regularly, private API endpoint, CIS benchmark (kube-bench)

10.3 NetworkPolicies

By default every pod can talk to every other pod. A **NetworkPolicy** selects pods by label and lists which traffic is allowed in (ingress) and out (egress). Once a pod is selected by any policy, everything not allowed is denied. Policies need a CNI that enforces them - Calico, Cilium or the cloud equivalents; Flannel alone does not.

> **Everyday example**
> A housing society gate: by default anyone walks in. After the committee adds rules - "only residents and registered delivery partners may enter Tower B" - the guard stops everyone else. The guard is the CNI; the rule book is the NetworkPolicy.

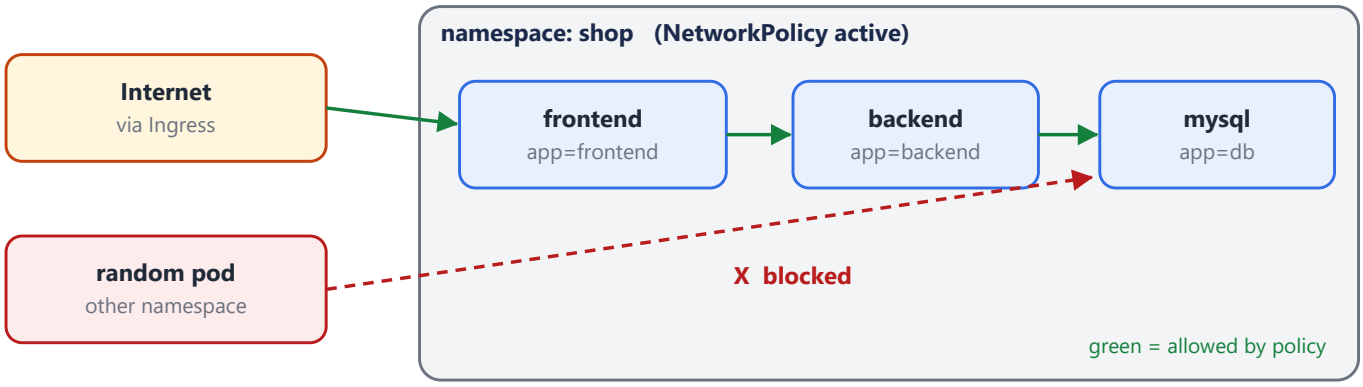


Figure 10.2 - Only the frontend may call the backend, and only the backend may call MySQL

netpol.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: backend-allow-frontend, namespace: shop }
spec:
  podSelector:
    matchLabels: { app: backend }
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: frontend }
      ports:
        - port: 80
```

10.4 CI/CD and GitOps

In **push-based CI/CD** a pipeline (Jenkins, GitHub Actions, GitLab CI, Azure DevOps) builds the image and runs `kubectl apply` or `helm upgrade` against the cluster. In **GitOps**, Git holds the desired state and an agent inside the cluster - **Argo CD** or **Flux** - pulls and applies it continuously, reverting manual drift. The manifests in Git are usually packaged as a Helm chart or Kustomize overlays - both are covered in depth in Class 11.

Everyday example
GitOps is like a building's approved floor plan kept at the municipal office. If someone secretly adds a wall, the inspector compares the building with the plan and removes it. To change the building, you first get the plan changed and approved (a pull request).

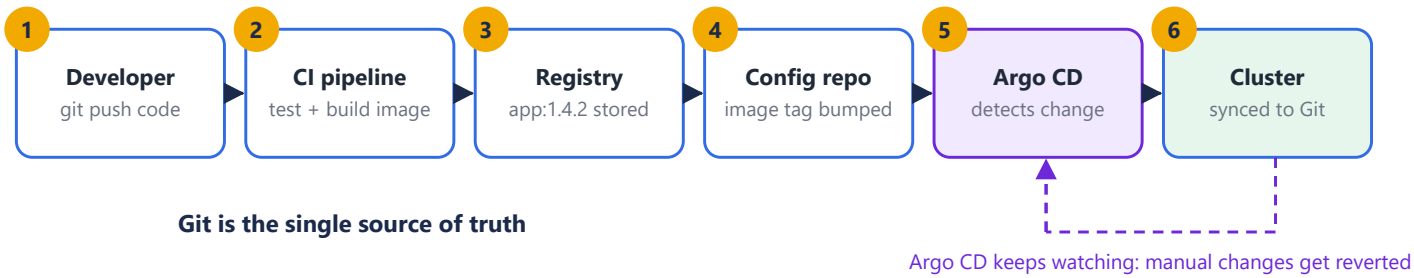


Figure 10.3 - GitOps flow: Git is the single source of truth and Argo CD keeps the cluster in sync

argocd-app.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: guestbook
  syncPolicy:
    automated: { prune: true, selfHeal: true }   # delete removed objects, undo manual edits
    syncOptions: ["CreateNamespace=true"]
```

10.5 Kubernetes in production

Topic	What you should know
Managed Kubernetes	EKS (AWS), AKS (Azure), GKE (Google): the cloud runs the control plane; you manage nodes and workloads
Self-managed	kubeadm (official), k3s (lightweight), Rancher, OpenShift (enterprise distribution)
High availability	3 control-plane nodes, etcd quorum, nodes across availability zones, anti-affinity, PDBs
Upgrades	One minor version at a time: control plane first, then nodes (drain, upgrade, uncordon)
Backup	`etcdctl snapshot save` for etcd; Velero for namespaces and volumes
CRDs	CustomResourceDefinitions add new object types, e.g. `Certificate`, `Application`, `PostgresCluster`
Operators	A CRD plus a controller that encodes human operational knowledge - install, backup, failover
Service mesh	Istio, Linkerd: mTLS, retries, traffic splitting and telemetry without code changes
Cost	Right-size requests, autoscale nodes, use spot instances for stateless work, delete unused volumes/LBs

Did you know?
Many cluster add-ons you met in this course are themselves Operators built on CRDs - cert-manager (`Certificate`), Argo CD (`Application`), Prometheus Operator (`ServiceMonitor`). Kubernetes is a platform for building platforms.

10.6 Live lab: security and GitOps

PowerShell - RBAC

```
kubectl create namespace dev
kubectl apply -f rbac.yaml
kubectl auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer      # yes
kubectl auth can-i delete pods -n dev --as=system:serviceaccount:dev:viewer    # no
kubectl auth can-i list secrets -n dev --as=system:serviceaccount:dev:viewer   # no
kubectl auth can-i --list -n dev --as=system:serviceaccount:dev:viewer
```

PowerShell - NetworkPolicy

```
minikube start -p secure --cni=calico --driver=docker
kubectl create namespace shop
kubectl -n shop run backend  --image=nginx:1.27 --labels=app=backend --port=80
kubectl -n shop expose pod backend --port=80
kubectl -n shop run frontend --image=busybox:1.36 --labels=app=frontend -- sleep 3600
kubectl -n shop run random   --image=busybox:1.36 --labels=app=random   -- sleep 3600

kubectl -n shop exec frontend -- wget -qO- -T 3 backend     # works
kubectl apply -f netpol.yaml
kubectl -n shop exec frontend -- wget -qO- -T 3 backend     # still works
kubectl -n shop exec random   -- wget -qO- -T 3 backend     # times out - blocked
minikube delete -p secure
```

PowerShell - Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side `
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd get pods -w                 # wait until all are Running

# admin password, then open https://localhost:8080 (user: admin)
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}"
kubectl -n argocd port-forward svc/argocd-server 8080:443

kubectl apply -f argocd-app.yaml              # in a second window
kubectl -n guestbook get pods                 # deployed from Git by Argo CD
kubectl -n guestbook scale deploy guestbook-ui --replicas=5
kubectl -n guestbook get deploy guestbook-ui -w   # selfHeal puts it back to 1
```

The password is base64-encoded - decode it as shown in Class 6. Remove Argo CD afterwards with `kubectl delete namespace argocd guestbook`.

10.7 Final project: WordPress + MySQL

Put the whole course together. Build it in a namespace called `blog`, store every file in a Git repository, and tick each item as you go:

Step	Objects	Concept from class
1	Namespace `blog` with a ResourceQuota and LimitRange	5
2	Secret `mysql-pass` with root and user passwords	6
3	MySQL StatefulSet (`mysql:8.0`) + headless Service + 1Gi volumeClaimTemplate	4, 7
4	ConfigMap with WordPress settings (DB host `mysql-0.mysql`, DB name)	6
5	WordPress Deployment (2 replicas) with readiness/liveness probes and requests/limits	3, 8, 9
6	ClusterIP Service + Ingress for `blog.local`	4, 5
7	HPA for WordPress (CPU 60%, 2-5 replicas) and a PDB (minAvailable 1)	9
8	NetworkPolicy: only WordPress pods can reach MySQL on 3306	10
9	Delete the MySQL pod and prove posts survive; roll out a new WordPress tag and roll back	3, 7
10	Bonus: convert everything into a Helm chart and deploy it with Argo CD	10, 11

Class 10 recap

- Every request: authentication -> authorization (RBAC) -> admission.
- Role/RoleBinding = namespace; ClusterRole/ClusterRoleBinding = cluster.
- Harden pods: non-root, read-only FS, drop capabilities, Pod Security `restricted`.
- NetworkPolicy = firewall for pods (needs Calico/Cilium).
- GitOps = Git is the truth; Argo CD pulls, syncs and self-heals drift.

Class 10 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is RBAC in Kubernetes?

Role-Based Access Control: an authorization mechanism where Roles and ClusterRoles define allowed verbs on resources, and RoleBindings and ClusterRoleBindings grant them to users, groups or ServiceAccounts. Everything not granted is denied.

Q2. Role vs ClusterRole?

A Role grants permissions within one namespace. A ClusterRole is cluster-wide and can cover cluster-scoped resources such as nodes, or be reused in many namespaces through RoleBindings.

Q3. RoleBinding vs ClusterRoleBinding?

A RoleBinding grants a Role or ClusterRole within a single namespace. A ClusterRoleBinding grants a ClusterRole across all namespaces.

Q4. What is a ServiceAccount?

A namespaced identity for processes running in pods. Pods get a short-lived token automatically mounted, used to call the API server with the permissions bound to that ServiceAccount.

Q5. How do you check if a user or ServiceAccount can perform an action?

`kubectl auth can-i <verb> <resource> -n <ns> --as=<user>` - for a ServiceAccount use `--as=system:serviceaccount:<ns>:<name>`; `--list` shows all permissions.

Q6. What are the stages an API request goes through?

Authentication (who), authorization (allowed, usually RBAC), then mutating and validating admission controllers (defaults, quotas, Pod Security, webhooks), and finally persistence in etcd.

Q7. What is a securityContext?

Pod or container settings that control privileges: runAsUser, runAsNonRoot, fsGroup, readOnlyRootFilesystem, allowPrivilegeEscalation, capabilities, seccomp and SELinux options.

Q8. What replaced PodSecurityPolicy?

Pod Security Admission, which enforces the Pod Security Standards (privileged, baseline, restricted) per namespace through labels. Kyverno or OPA Gatekeeper add custom policies.

Q9. What is a NetworkPolicy?

A namespaced object that controls pod-level ingress and egress traffic using pod selectors, namespace selectors, IP blocks and ports. Pods selected by a policy deny all traffic not explicitly allowed.

Q10. Does every cluster enforce NetworkPolicies?

No. Enforcement depends on the CNI plugin. Calico, Cilium and most cloud CNIs support it; plain Flannel does not, so policies would have no effect.

Q11. How do you create a default-deny policy for a namespace?

A NetworkPolicy with an empty podSelector (`{}`) and policyTypes Ingress (and Egress) but no rules. It selects all pods and allows nothing, and specific allow policies are added on top.

Q12. Why should containers not run as root?

If an attacker breaks out of a root container they may gain root on the node. Running as a non-root user with dropped capabilities, no privilege escalation and a read-only file system limits the damage.

Q13. What are admission webhooks?

HTTP callbacks the API server calls during admission. Mutating webhooks can change objects (inject sidecars, add defaults); validating webhooks can reject them. Kyverno, Gatekeeper and Istio use them.

Q14. How do you make sure only trusted images run in the cluster?

Scan images in CI (Trivy, Grype), sign them (cosign/Sigstore), and enforce policies with Kyverno or Gatekeeper that allow only approved registries, pinned tags or digests, and verified signatures.

Q15. What is Kubernetes audit logging?

The API server can record who did what and when for every request, according to an audit policy. Audit logs are essential for security investigations and compliance.

Q16. What is GitOps?

An operating model where the desired state of the system is stored in Git and an in-cluster agent (Argo CD, Flux) continuously pulls and reconciles it, giving audit history, easy rollback via git revert, and drift correction.

Q17. What is Argo CD?

A declarative GitOps continuous-delivery tool for Kubernetes. It watches Git repositories, compares them with live cluster state, shows differences in a UI, and syncs automatically or on demand.

Q18. What are CRDs and Operators?

A CustomResourceDefinition extends the Kubernetes API with new object types. An Operator is a controller that watches those custom resources and automates operational tasks such as installing, backing up, scaling and failing over complex applications.

Q19. How do you upgrade a Kubernetes cluster?

Read release notes and deprecations, back up etcd, upgrade one minor version at a time - control plane first, then worker nodes one by one with drain, upgrade kubelet, uncordon - using PDBs to keep apps available. Managed services automate much of this.

Q20. What are managed Kubernetes services and why use them?

EKS, AKS and GKE run and patch the control plane, integrate with cloud IAM, load balancers and storage, and offer node autoscaling. Teams focus on workloads instead of operating etcd and the API server.